

XPression 実務ハンズオン 手順書

DekoBokoTech / XPression実務ノート

本書は、同梱のサンプル素材を使って XPression の実務を手を動かして学ぶための手順書です。各ハンズオンのフォルダ（01～03）に対応する素材が入っています。

※サンプル素材はすべて自作のダミーです。実在のチーム・ロゴ・色は使用していません。

収録ハンズオン

No.	タイトル	レベル	目安	対応フォルダ
1	壊れないローワースードを作る	中級	30分	01_LowerThird
2	スコアボードを作る	中級～上級	40分	02_Scoreboard
3	DataLinqで画像を差し替える	中級	25分	03_DataLinq_ImageSwap
4	簡易ゲームクロックを作る	中級	30分	04_GameClock
5	CSV／Excelでスコアや名前を流し込む	中級	35分	05_DataLinq_Practice
6	Staggerで一文字ずつ出す見出し	中級	25分	06_Stagger_Text
7	Transitionで出入りを設計する	中級	25分	07_Transition
8	スタッフロール（Roll）を作る	中級	30分	08_StaffRoll
9	Maskで見せる範囲を作る	中級	25分	09_Mask
10	Gradientで読みやすい色面を作る	初級	20分	10_Gradient
11	現在時刻・カウントダウンを表示する	中級	30分	11_Clock_Countdown

各ハンズオンの素材は、対応フォルダの中に入っています。手順を読みながら、その素材を使って進めてください。

1. 壊れないローワースードを作る

ネイティブ機能とVisual Logicの分担

4 | 第12章 | レベル：中級 | 目安 30分 | 素材フォルダ：01_LowerThird

ねらい

名前と肩書きの2段ローワースード。短い名前でも長い名前でも背景が文字に合わせて気持ちよく収まり、いくら長くても安全エリアからはみ出さない上限を持つ。肩書きが空のときは行を出さず名前だけできれいに見せ、Online / Offline / Take で安定して出し隠しできる。

STEP 1 器を作る（背景とText Objectの親子）

新しいSceneを作り、名前用のText Objectを置き、その最初の子（First Child）として背景を持たせる。この親子関係が後のAuto Scaleで効く。背景はMaterialで色を持たせる（色は見た目ではなく値として持たせると、チームカラーへの差し替えが楽）。肩書き用のText Objectも置き、名前は Name、肩書きは Title のように役割が分かる名前を付けておく。

STEP 2 ネイティブで幅の土台を作る（ここが本命）

名前のText Object の Tabs & Options タブで3つを設定する。①Auto Squeeze：Enabled にして Max Width に上限幅を入れ、Scaling は Width Only（横だけ詰める）。②Auto Scale：Enabled・Target = First Child・Mode = Width & Height にすると、親の詰まりに子の背景が追従する（背景幅をVisual Logicで計算する必要はない）。③Character Limits：Soft Limit で警告（文字が赤くなる）、Hard Limit で文字数を頭打ち（最後の保険）。これで幅まわりは完成、Visual Logicは一切使わない。

ヒント：Max Width・Hard Limit

の具体値は安全エリアとフォント依存。実機で最長の名前を入れて決める。

STEP 3 Visual Logicの出番①：肩書きが空なら、行を出さない

比較と Not

の考え方で「肩書きが空ではないときだけ肩書きの行を表示する」という条件を作り、肩書きText Object の Visible につなぐ。空のときは隠す。名前を中央に寄せたいときは、名前Text Object の Position (X/Y/Z)

につないで動かす。オペレーターが毎回手で直さなくても、テンプレートが整えてくれる。

STEP 4 設計の判断：読めなくなる手前で切り替える（Visual Logicの出番②）

Width Only の横詰めは便利だが、詰めすぎると文字が縦長になり、ある所から急に読みにくくなる（「収まる」と「読める」は別）。名前の長さをVisual Logicで見て、決めたしきい値を超えたら二行折り返しなどに切り替える。折り返しそのものはネイティブの Word Wrap が担当し、Visual Logicは「どこで切り替えるか」の判断だけを持つ。

ヒント：切り替えのしきい値は文字数で近似する。解像度・セーフティゾーン・フォント依存なので実機で決める。

STEP 5 出す・隠すを整える

ローワーサードをScene単位で扱う。送出はTake（テンキーの＋キー、またはRossTalkのTAKE）、消すときはOffline（テンキーの－キー、またはSEQO）。短い名前・長い名前・極端に長い名前・肩書き無しの各パターンをリハで実際に出し、どの状態でも崩れず読めるか確認する。差し替えたら、まずプレビューでAuto Squeezeと背景の追従が崩れていないかを確認し、問題なければ再TAKEで送出に反映する。

本番前チェックリスト

- ☐ いちばん長く想定される名前で、Auto Squeezeが効いて安全エリアからはみ出さない
- ☐ 背景がAuto Scaleで名前に追従し、余白が破綻していない
- ☐ 極端に長い名前で、Hard Limitが最後の保険として効く
- ☐ 横詰めが効きすぎて読みにくくなる手前で切り替えが入る（収まる≠読める）
- ☐ 肩書き無しするとき、間延びせず名前だけきれいに収まる
- ☐ 送出中に名前を差し替えても、詰めと背景の追従が破綻しない

2. スコアボードを作る

Visual Logicが本領を出す場所

4 | 第13章 | レベル：中級～上級 | 目安 40分 | 素材フォルダ：02_Scoreboard

ねらい

2チームのスコアボード。ホームとアウェイのチーム名・色・スコア・ピリオド表示。勝っている側がひと目で分かるリード強調。スコアが動いたときは変わった数字が軽く強調され、登場と退場はなめらかに。

STEP 1 器を作る（スコアボードのオブジェクト構成）

新しいSceneに部品を並べる。チーム名・スコアの数字・背景パネル・リードマーカー（グローや矢印）を、ホーム側とアウェイ側で左右対称に持つ。HomeName / HomeScore / AwayName / AwayScore、リードマーカーは HomeLead / AwayLead
のように、左右と役割が一目で分かる名前を付けると結線で迷わない。

STEP 2 スコアを表示用に整える

スコアは数値だが、画面に出すときは見せ方を整える（1桁を「7」と出すか「07」と出すか、3桁で桁あふれしないか）。第4・6章の Format の実物応用。数値はそのまま出さず、Format で表示用の形に整えてから表示につなぐ。「外部データほど、表示の直前で型を整える」をそのまま適用する。

ヒント：

桁数の想定（最大何点まで）とゼロ詰めの有無は競技・運用で決め、実機で桁あふれの挙動を確認する。

STEP 3 Visual Logicの出番①：リードを強調する

この章の中心。Greater Than でホームとアウェイのスコアを比較し、大きい側のマーカーを Visible / Color で強調する（勝っている側を明るいチームカラー、負けている側を落ち着いた色に）。ネイティブは「どちらが大きいか」を判断できないので、ここはVisual Logicの仕事。そして同点の分岐を最初から入れる（放っておくと「両方が勝っている」ように見える事故になる）。

ヒント： 比較・色切り替え・同点判定の具体的なFunction Blocks結線は、実機で組んだものを正とする。

STEP 4 Visual Logicの出番②：チームカラーを出し分ける

対戦カードごとにチームが変わり、色も変わる。色を値として持たせておけば、チームの選択を入力にして対応するチームカラーを背景やマーカーに流し込める（Selectorsの考え方）。「どの色にするか」を決めるのがVisual Logic、「その色で表示する」のがネイティブ（Material）。

STEP 5 動きをネイティブで付ける

登場と退場は Scene Director と Transition でなめらかに、順序は Sequencer で管理する。スコアが入った瞬間の強調は、変わった数字を軽く光らせる・弾ませる（常時の細かな動きは Continuous Animation）。「スコアが変わったこと」を Visual Logic で検知し、その瞬間に強調を出す組み方もできる（変化の検知は Timers / Counters の考え方）。値が動く瞬間に表示が破綻しないか必ず確認する。

STEP 6 発展：スコアに比例したバーを作る（任意）

点差を視覚的に見せたいときは、スコアの値を入力にバーの長さや位置を計算する（Math の応用）。ただし伸ばしっぱなしにせず、最大値で頭打ちにする（ローワーサードの幅と同じ考え方）。数字だけで足りる現場も多いので、必要なときに足す部品として捉える。

本番前チェックリスト

- ☐ ホームリード・アウェイリード・同点の3状態すべてで、強調が正しく出る
- ☐ 同点のとき、両方が勝っているように見えていない
- ☐ チームを差し替えたとき、色が正しく出し分けられる
- ☐ スコアが2桁から3桁になっても、桁あふれせず読める
- ☐ スコアが入れ替わった瞬間・同点になった瞬間に、表示が破綻しない
- ☐ 登場・退場がなめらかで、送出の順序が乱れない

3. DataLinqで画像を差し替える

Dynamic Material と Material Path（画像・ロゴの自動差し替え）

5 | 第3章 | レベル：中級 | 目安 25分 | 素材フォルダ：03_DataLinq_ImageSwap

ねらい

チームロゴ・選手写真・スポンサー素材などの画像を、毎回手で貼り替えずに切り替えられるテンプレートを作ります。表示する画像を「ファイルの置き場所（パス）」で決めるようにし、さらに Widget やデータの値を変えるだけで別の画像へ入れ替わる状態を目指します。対戦カードや出演者が変わっても、画像を貼り替える手間がなくなります。

STEP 1 素材ファイルを整理する

まず、差し替えたい画像をあとで指定しやすい場所に整理します。プロジェクトフォルダの中に「Images」フォルダを作り、用途ごとにサブフォルダを分けて、次のようなファイルを用意します。

【準備するフォルダとファイルの例】

Images/Teams/

- TokyoTeamLogo.png
- OsakaTeamLogo.png
- NagoyaTeamLogo.png

Images/Players/

- Player_01_Photo.png
- Player_02_Photo.png

Images/Sponsors/

- Sponsor_Main.png

Images/Backgrounds/

- Arena_Background.png

大事なポイント：チームロゴは「チーム名+Logo.png」の形でファイル名を統一します（TokyoTeamLogo.png / OsakaTeamLogo.png …）。こうしておくと、⑤で Widget の値（TokyoTeam / OsakaTeam …）を変えるだけで、対応するファイルへ自動で切り替わります。

ヒント：フォルダ名・ファイル名は半角英数字で統一するとパス指定でトラブりにくいです。ファイル名の付け方（値+固定語）を先に決めておくと、後の自動切り替え（⑤）がそのままハマります。

STEP 2 画像を表示するオブジェクトを用意する

画像を映すためのオブジェクトを置きます。チームロゴやスポンサー画像なら、四角い面に画像を貼れる Quad Object が分かりやすいです。すでにロゴ枠などがある場合はそれを使ってOK。あとで指定しやすいよう、オブジェクトには「HomeLogo」など役割が分かる名前を付けておきましょう。

ヒント：「画像を貼る面」を用意するイメージ。まだ固定の画像が入っていても大丈夫です（次で動的に切り替えます）。

STEP 3 マテリアルを Dynamic Material に切り替える

画像を出したいオブジェクトを選択し、Object

Inspector（設定パネル）を開きます。「DataLinq」タブへ移動し、「Select Material

Source」で「Dynamic Material」を選びます。これで、このオブジェクトの画像が『固定』ではなく『後から指定して差し替えられる』状態になります。

ヒント：Dynamic Material＝『表示する画像を、パスやデータで動的に決められるマテリアル』。ここが画像差し替えの心臓部です。

STEP 4 Material Path で表示する画像を指定する

Dynamic Material Properties 中の「Material Path」に、表示したい画像ファイルの場所（パス）を入力します。プロジェクトフォルダからの相対パスで書くのが基本です。

記入例：

- ・チームロゴ → Images/Teams/TokyoTeamLogo.png
- ・選手写真 → Images/Players/Player_01_Photo.png
- ・スポンサー → Images/Sponsors/Sponsor_Main.png
- ・背景 → Images/Backgrounds/Arena_Background.png

ここで指定したファイルが、そのオブジェクトに表示されます。

【重要：編集画面（Layout）は市松模様でOK。送出側で表示される】

編集中の Layout ビューでは Dynamic Material はプレビューされず、青と黄色の市松模様（プレースホルダー）が表示されます。これは正常な挙動で、パスが間違っているわけではありません。実際の画像は送出（シーケンサー／Take＝オンエア）側で正しく表示されます。まずは Sequence で表示を確認してください。

【送出（Sequence）側でも画像が出ないときだけ、パスを見直す】

Sequence でも市松のまま／画像が出ない場合は、パスが解決できていません。次を確認します：

- ・パスの前後に "（ダブルクォート）が付いていないか。「パスのコピー」で貼り付けると "C:/…/TokyoTeamLogo.png" のように付くことがあり、残っていると解決できません。必ず削除。
- ・区切りが¥ではなく / になっているか。
- ・フォルダ名・ファイル名に日本語や全角が混じっていないか（半角英数字が安全）。
- ・指定したファイルが実際にその場所にあるか、名前・拡張子（大文字小文字含む）が一致しているか。
- ・相対パスの場合、プロジェクトを保存し、Images フォルダが保存先プロジェクトの下にあるか。（編集画面で位置やサイズを合わせたいときは、作業中だけ Static Material に同じ画像を割り当て、送出前に Dynamic Material へ戻すと確認しやすいです。）

【パスをコピーする方法（Windows）】

- ・エクスプローラーで対象ファイルを Shift+右クリック → 「パスのコピー」を選ぶ。
- ・フォルダのパスだけ欲しいときは、アドレスバーをクリックしてコピー。

コピーされるのは絶対パスです。Material Path を相対にしたいときは、プロジェクトフォルダより下の部分（例：Images/Teams/…）だけを残して貼り付けます。区切りは / でOKです。

- ・重要：「パスのコピー」で貼り付けると、パスの前後に "（ダブルクォート）が付くことがあります（例："C:/…/TokyoTeamLogo.png"）。この " は必ず削除してください。残っているとパスを解決できず、市松模様のままになります。

ヒント：絶対パス（D:/… など）を使う場合は、本番機でも同じパスで解決できるか必ず確認を。相対パスの方が別PCへ移しても崩れにくく安全です。

STEP 5 Widget (Text List) の値でファイル名を切り替える (具体的な書き方)

ここが自動切り替えの肝です。まず切り替え用の Widget を作ります。

Display メニュー → Widgets パネルの『New Widget』 → 『Text List』を選びます (=テキストを切り替えられるウィジェット。時刻なら Clock Timer、数値なら Counter)。

できた『TextList1』は、分かりやすい名前 (例: TeamNameForFile) に変更しておきます。

次に Widget Properties (プロパティ) を開き、『Add』で切り替えたい値を登録します。

例: TokyoTeam / OsakaTeam / NagoyaTeam …

※『Allow manual entry of text』にチェックすると、一覧に無い値もその場で手入力できます。

そして Material Path のファイル名部分に、この Widget を差し込むマクロ @W:ウィジェット名@ を書きます。

記入例:

Images/Teams/@W:TeamNameForFile@Logo.png

【手入力しない方法: 右クリックで挿入】

マクロは手で打たなくてもOKです。Material Path の入力欄を右クリック → 『Insert

Lookup』 → 『Widgets』 → 作成した

Widget名 (例: TeamNameForFile) を選ぶと、@W:TeamNameForFile@ が自動で挿入されます。

同じ右クリックメニューから『Published Text Objects』『All Text Objects』を選べば、テキストオブジェクトの値も同じように差し込めます。打ち間違い (スペルミス) を防げるので、こちらが確実です。

@W:TeamNameForFile@ の部分が、いま選ばれている値に置き換わります。

- 選択が TokyoTeam のとき → Images/Teams/TokyoTeamLogo.png
- OsakaTeam に切り替えると → Images/Teams/OsakaTeamLogo.png

Widgets パネルのドロップダウンや Prev / Next

で値を切り替えると、表示ロゴが自動で入れ替わります。

ヒント: 値の種類で使うWidgetが違います: 文字 (チーム名など) =Text

List、数値カウント=Counter、時刻=Clock

Timer。コツはファイル名を『値+固定語』で統一すること (TokyoTeamLogo.png /

OsakaTeamLogo.png…)。外部データで切り替えるなら @W:名前@ の代わりに %DataLinqキー名% を使います。

STEP 6 本番前に確認する

想定するすべての画像で、パスが正しく解決できて画像が表示されるかを確認します。特に次の3点をチェック：

- ・ 相対パスが別PC（本番機）でも通るか（保存場所や共有ドライブの割り当て違いに注意）
- ・ Widget／データの値を変えたら、本当に画像が切り替わるか
- ・ 指定した画像が見つからない（空・欠番）ときに大きく崩れないか

リハーサルで実際に何パターンか切り替えてみると安心です。

ヒント：「本番機で画像が出ない」の多くはパスの解決ミス。素材はプロジェクトに同梱し、相対パスで指定しておくことで事故が減ります。素材の置き場所が毎回変わる現場では file path 指定は慎重に。

本番前チェックリスト

- ☐ 相対パスで、別のPC（本番機）でも画像が表示される
- ☐ Widget（@W:名前@）またはDataLinqキー（%キー名%）の値を変えるだけで画像が切り替わる
- ☐ ファイル名を『値＋固定語』で統一してある（TokyoTeamLogo.png など）
- ☐ 絶対パスを使う場合、本番機でも同じパスで解決できることを確認した
- ☐ 指定した画像が無いとき（空・欠番）に表示が大きく崩れない
- ☐ 対戦カード・出演者の差し替えを、手作業なしで回せる

4. 簡易ゲームクロックを作る

Clock Timer Widgetでカウントダウン表示

4 | 第14章 | レベル：中級 | 目安 30分 | 素材フォルダ：04_GameClock

ねらい

12:00から00:00へ向かってカウントダウンする簡易ゲームクロック。Text Objectは表示先で、時間の値はClock Timer Widgetが持つ。Start / Stop / ResetをManualで確実に操作でき、Online・再Takeでも意図しないResetや再Startが起きないことを確認する。

STEP 1 作り方を3つに切り分ける

時計・タイマー表示は目的で道具が変わる。①シンプルなカウントダウン→Clock Timer Widget（今回はこれ）。②残り時間で色を変えるなどの演出連動→Visual Logic。③外部スコアボード等の公式時計を表示→DataLinq。ここでは最も分かりやすいClock Timer Widgetで作る。Visual Logicを無理に最初から使わない。

STEP 2 テスト用Sceneを作る

新規Sceneを作り、GameClock_Test など分かる名前を付ける。この段階では背景・装飾・ロゴ・スコアは入れない。まず確認したいのは見た目ではなく『時計が正しく表示され、Start / Stop / Resetできるか』。

STEP 3 表示先のText Objectを置く

Scene上にText Objectを1つ置き、仮で「12:00」と入れる（位置・サイズ・Font・Materialの確認用）。名前はClock_Textのように役割が分かる名前にする。重要：この仮文字を打っただけでは動くタイマーにはならない。Text Objectは時間を作る部品ではなく、時間の値を表示する表示先。

STEP 4 Clock Timer Widgetを作る

メニューのDisplay > Widgetsを開き、New WidgetからClock Timerを作る。名前はGameClock_Timerのように用途が分かる名前に（後でText Object側から選ぶときに効く）。Widget画面は『Timerを作る場所』であり、同時に『Timerの状態を見て操作する場所』でもある。

ヒント：Widgetが増えるとどのTimerをどのTextにつないだか分からなくなる。最初から用途の分かる名前を付けておく。

STEP 5 ModeをTimerにする

作ったClock Timer WidgetのPropertiesでModeをTimerにする。Clock＝『今の時刻』を扱う考え方、Timer＝『任意の開始値から進める／戻す』考え方。12:00から00:00へ減らしたいのでTimerが中心。

STEP 6 Start At / Stop At / Direction を決める

12分→0分のカウントダウンなら、Start At=12:00、Stop At=00:00、Direction=Down。暗記すべきは設定名ではなく『どこから始まる／どこで止まる／上がるか下がるか』の3つ。入力形式や項目名はバージョン・画面で異なるので実機で確認する。

ヒント： この3つを間違えると、表示形式が合ってもタイマーとして使いにくくなる。

STEP 7 Formatを決める（まずはNN:SS）

FormatはTimerの値をどんな文字列で出すかの設定。NN:SS / HH:NN:SS / NN:SS.ZZZなどが扱える。簡易ゲームクロックはNN:SS（例 12:00 / 05:30 / 00:10）が分かりやすい。小数表示（.ZZZ）は桁が増え、Fontによっては数字の幅が揃わず揺れて見えるので、最初はNN:SSが安全。

STEP 8 Start / Stop / Reset を Manual にする

最初の作例ではStart / Stop / ResetのMethodをManualにする（Operatorが明示的に操作）。When Onlineにすると出した瞬間に動き出し、原因の切り分けがしにくい。注意：『Manualに設定する』ことと『実際にStartする』ことは別。StartはあとでWidget側の操作で実行する。Ctrl+1～9などショートカット割り当てはKeyboard Mappingと競合しないか実機で確認。

STEP 9 Text ObjectのData SourceにWidgetをつなぐ

Clock_Textを選び、Text ObjectのData Sourceタブを開く。Data SourceとしてWidgetを選び、一覧からGameClock_Timerを選ぶ。これでClock_Textにタイマーの値が表示される。直接『12:00』と打ちっぱなしにしない（それは固定表示）。表示されないときは、対象Text選択・Data SourceでWidget選択・正しいWidget指定・Format・Font/Material/位置を順に確認。

STEP 10 Main Viewportで表示を確認する

接続できたらMain Viewportで値が出ているか確認。ここで見るのはデザインではなく『Widgetの値が表示されているか／Formatが意図通りか／文字が範囲に収まるか』。動きが見えないときはViewport上部のShow or Hide Continuous Animations and Other Effectsなどの表示更新状態も確認。表示されている＝Startしている、ではない点に注意。

STEP 11 Widget側のManual操作でStart→Stop→Reset

Widget側の操作でStartし、12:00 → 11:59 → 11:58 と減れば表示接続は成功。動かないときはText Objectのデザインより先に『WidgetがStartしているか／MethodがManualか／Start操作を実行したか／Start At・Directionが意図通りか』を確認。次にStopで保持、Resetで開始値へ戻るかを確認。Resetは強力（本番中の誤操作で意図しない値に戻る）。『いつ／誰がResetしてよいか』『自動Reset（Reset on timer等）が有効になっていないか』まで確認する。

STEP 12 OnlineにしてTake/Offline/再Takeを確認する

Layoutで確認できたらSceneをOnlineにして送出時の挙動を見る。Main Viewportで動いても送出時に同じとは限らない。確認：Onlineにしたとき時計は止まっているか／Manual Startまで動かないか／Offlineで止まる・保持・Resetのどれになるか／再Take時に意図しないReset・再Startが起きないか。再Take時の値保持/ResetはScene属性やSequencer・Widgetの設定に依存するので必ず実機で確認し、Operatorが説明できる状態にしておく。

本番前チェックリスト

- ☐ 3つの作り方のうち、簡易＝Clock Timer Widgetを選んでいる
- ☐ Text Objectは表示先、時間の値はClock Timer Widgetが持つ、と分けて作っている
- ☐ Start At / Stop At / Direction が意図通り（どこから・どこで止まる・増減）
- ☐ Manual設定と実際のStart操作を混同していない（表示≠Start）
- ☐ Reset のタイミングと権限、自動Resetの有無を確認した
- ☐ Online・Offline・再Takeで意図しないReset/再Startが起きないことを確認した

5. CSV／Excelでスコアや名前を流し込む

DataLinqとWidgetsで手入力を減らす

5 | 第3・4章 | レベル：中級 | 目安 35分 | 素材フォルダ：05_DataLinq_Practice

ねらい

外部の表（CSV/Excel）から選手名・スコア・チーム情報をText Objectへ流し込み、表を書き換えるだけで表示が変わる状態を作る。データソースは『誰が編集し、どう壊れるか』で選び、失敗時の見え方まで想定する。

STEP 1 DataLinqの共通構造を押さえる

DataLinqは『外部データを取り込み→キー（列や項目）で参照→Text ObjectやMaterialに表示』という共通構造。形式（CSV/Excel/JSON）が違ってても骨格は同じ。まず『何の値を、どのキーで、どこに出すか』を決める。

STEP 2 まずCSVで動かす

単純で追いやすいデータはCSVが向く。同梱の サンプルデータ_チーム.csv（TeamName / Abbr / ColorHex / LogoFile）や サンプルデータ_名前と肩書き.csv（Name / Title）を使う。DataLinqでCSVをデータソースとして読み込み、列名（キー）をText ObjectのData Sourceに割り当てる。文字コード・区切り・ヘッダー行の有無でつまずきやすいので、まず1件表示して確認。

ヒント： CSVは半角英数字の列名・UTF-8がトラブりにくい。1行目（ヘッダー）がキーになる。

STEP 3 行を切り替えて表示を変える

読み込んだ表の『どの行を表示するか』を切り替えると、名前やスコアが差し替わる。ローワースードなら1件ずつ、スコアボードならHome/Awayの2件を割り当てる。表（CSV）側を書き換えて再読み込みし、表示が追従するか確認する。

STEP 4 編集しやすさが要るならExcel、構造化ならJSON

現場担当者が頻繁に手編集するならExcelが選ばれやすい（ただし、どのシート・範囲・保存形式を読むかの読み取り方式を実機で確認）。項目が入れ子・外部システム連携ならJSON。断定より『現場で確認すべき観点』で選ぶ。

STEP 5 本番挙動（更新・失敗時）を決める

Live Update（自動更新）は便利だが全項目で使うものではない。Poll every（取得間隔）は短ければよいわけではない。Return Empty on Failure（失敗時に空を返す）で、データが取れないときの見え方が決まる。取れないとき前の値が残るのか空になるのかを想定して選ぶ。

ヒント：『空データ時にどう見えるか』を必ず一度作って確認。Prepend/Appendを使う場合は空データ時の挙動に注意。

STEP 6 Widgetsで切り替えも足す（任意）

手入力を減らす仕上がりとしてWidgetsが使える。Text Listで選択肢を持つ、Counterでスコアを増減、Clock Timerで時刻。Widgetは作っただけでは表示に出ず、Text ObjectのData Sourceに割り当てて初めて効く。DataLinq（外部データ）とWidget（現場操作の値）は分けて考える。

STEP 7 本番前に確認する

データソースは『誰が編集し、どう壊れるか』で選ぶ。確認：想定する全データで正しく表示されるか／文字数が多い行でも崩れないか／データが取れない・空・欠番のときに大きく破綻しないか／編集ルール（列を消さない等）を担当者が守れるか。

本番前チェックリスト

- ☐ 『何の値を・どのキーで・どこに出すか』を先に決めている
- ☐ CSVで1件表示→行切り替えで差し替わることを確認した
- ☐ 形式（CSV/Excel/JSON）を『編集頻度・構造・連携』で選んでいる
- ☐ Live Update / Poll every / Return Empty on Failure の挙動を決めた
- ☐ 空データ・欠番時の見え方を確認した
- ☐ Widgetを使う場合、Data Sourceへ割り当てて表示に反映している

6. Staggerで一文字ずつ出す見出し

現場で読めるテキスト演出

3 | 第2章 | レベル：中級 | 目安 25分 | 素材フォルダ：06_Stagger_Text

ねらい

文字が少しずつずれて登場する見出し。設定を作る→Scene

Director上の対象へ配置→再生して確認、の3ステップで、Timing Offsetsで『文字ごと／単語ごと／行ごと』のずれを設計する。差し替わっても成立する演出にする。

STEP 1 3ステップの流れを押さえる

Stagger Animationは①アニメーションの設定を作る②その設定をScene

Director上の対象オブジェクト（Text

ObjectやGroup）へ配置する③再生して見え方を確認する、の3つ。Scene ManagerでSceneやScene Groupを選ぶのは、動かしたいオブジェクトを含むシーンを選ぶため。

STEP 2 対象と名前を決める

動かすText ObjectをScene Directorに追加する。NameとDescriptionは現場運用のために使う（後から見て何の動きか分かるように）。リアルタイムでは、きれいさだけでなく『あとから見ても分かる状態』が大事。

STEP 3 Trackで『何を動かすか』を決める

Trackは動かすプロパティ。Position.X/Y/Z（位置）、Rotation（回転）、Scale（拡大縮小）、Alpha（透明度）など。登場なら

Alpha+Position（フェードしながら少し動く）が定番。動かす対象だけTrackを持たせる。

STEP 4 Keyframeで動きの形を作る

Keyframeは動きの形を決める。Clip Length（そのTrackの変化にかかる長さ）とTotal Duration（全体尺）は分けて考える。尺を変えたら Recalculate Keyframe Positions で位置がずれていないか確認。

ヒント：Track Controlsは作業中の事故（誤って別Trackを動かす等）を減らすために使う。

STEP 5 Timing Offsetsでずれを作る（ここが中心）

Stagger Animationの中心はTiming Offsets。Character（文字ごと）／Word（単語ごと）／Line（行ごと）／Paragraph（段落ごと）で、どの単位でずらすかを決める。日本語見出しなら Character や Word でのずれが読みやすいことが多い。PivotはScaleやRotationで特に効く（回転・拡大の中心）。

STEP 6 再生して読めるか確認する

実際に再生し、演出として気持ちよいかだけでなく『読めるか』を確認する。速すぎると文字が追えない。文字数が変わる差し替え（長い名前・短い名前）でも破綻しないかを見る。

本番前チェックリスト

- ☐ 設定を作る→対象へ配置→再生確認、の3ステップで進めた
- ☐ Name / Description を後から分かるように付けた
- ☐ 動かすTrack（Alpha/Position等）を必要な分だけ持たせた
- ☐ Clip LengthとTotal Durationを分けて考え、尺変更後に再計算で確認した
- ☐ Timing Offsetsのずれ単位（Character/Word/Line/Paragraph）を選んだ
- ☐ 文字数が変わっても読める速度・崩れないことを確認した

7. Transitionで出入りを設計する

正しく消えて、正しく更新できるSceneにする

3 | 第8章 | レベル：中級 | 目安 25分 | 素材フォルダ：07_Transition

ねらい

Sceneの登場（In）だけでなく退場（Out）と更新（Back-to-Back）まで設計し、本番で残り方が読みにくくならないようにする。Layerと重なり順、外部制御での消え方まで想定する。

STEP 1 Transitionが設計するものを理解する

Transitionは単なるエフェクトではなく、Sceneの出入りを破綻させないための設計項目。In（出る）だけでなく Out（消える）・更新（連続で内容が変わる）まで含む。

STEP 2 In / Out をまず作る

Transition In / Out を設定する。Cut / Dissolve / Push / Distort などを用途で使い分ける。まずはInとOutの両方を必ず作る（Outを設計しないと本番で残り方が読みにくい）。

STEP 3 Durationは現場テンポで決める

Durationは見た目の好みではなく現場テンポで決める。常駐テロップは自然にOut、緊急表示は即座に消せる経路、演出SceneはOutまで含めて見せ方を作る、スコア更新系は次の情報を邪魔しない長さに。

STEP 4 更新（Back-to-Back）を設計する

同じSceneで内容が連続更新される（スコアや情報更新）なら、Back-to-Backで『更新の見え方』を設計する。毎回In/Outを打つのか、中身だけ差し替えるのかを決める。

STEP 5 Scene DirectorとTransition Logicを分ける

Scene Directorは動きの中身を作る場所。Transition Logicは条件に応じてどのScene Directorを使うかを決める場所。単純なIn/Outで足りるならそれで済ませ、必要な複雑さに留める（作り込みすぎるとデータが読みにくなる）。

ヒント：

本番前修正・別担当・複製・Layer変更・外部制御変更が起きる前提で、必要以上に高度にしない。

STEP 6 LayerとFramebuffer・外部制御まで見る

動きだけでなく重なり順（Layer / Framebuffer）まで見る。別Layerの状態に反応させるならScene Triggersを検討。外部制御（GPI / RossTalk / PBus等）で消す場合は、Transitionの想定と実際の消え方が一致しているか確認。

STEP 7 本番前の確認順序

In だけを見ず、更新・Out・Layer・外部制御まで含めて確認する。確認：出たあと正しく消えるか／連続更新が破綻しないか／重なり順が意図通りか／外部制御で消したとき想定通りか。

本番前チェックリスト

- ☐ In と Out の両方を作った（Outを設計している）
- ☐ Durationを現場テンポ・用途で決めた
- ☐ 連続更新があるならBack-to-Backを設計した
- ☐ Scene DirectorとTransition Logicの役割を分けている
- ☐ Layer/Framebufferの重なり順を確認した
- ☐ 外部制御での消え方が想定と一致することを確認した

8. スタッフロール（Roll）を作る

Scene GroupでRoll/Crawlを設計する

3 | 第9・10章 | レベル：中級 | 目安 30分 | 素材フォルダ：08_StaffRoll

ねらい

スタッフ名・役職が縦に流れるエンドロール。『どう動かすか』より『どう直すか・どう運用するか』を先に決め、Global Marginsで安全に見える範囲を保ち、Loopの止め方まで設計する。

STEP 1 用途を先に決める

Scene Groupを作る前に『何を、どう流すか』を決める。スタッフロールならスタッフ名・役職を表示するSceneをまとめる。スポンサーRollならロゴ入りSceneをまとめる。地味だがここが一番大事。作るときは『どう動かすか』より『どう直すか（本番前の修正・別担当・再利用）』を先に考える。

STEP 2 Scene GroupとSequenceのどちらで作るか選ぶ

Scene Group＝デザインのまとまりを優先、テンプレート構造に近い場所で管理。Sequence側（Take Item Group）＝内容変更が多い運用向き、送出の並びで管理。今回はScene Groupで作る。どちらでも『あとから直す人が理解できる構造』にする。

STEP 3 現場で分かる名前を付ける

Scene Groupにも中のSceneにも、後から見て分かる名前を付ける（例：EndRoll_Staff）。名前が曖昧だと、修正・再利用の段階で誰も触れなくなる。

STEP 4 EffectでRoll（縦）かCrawl（横）を選ぶ

縦に流すのがRoll、横に流すのがCrawl。スタッフロールはRoll。Effectで種類を選び、Directionで視線の流れ（下から上など）を決める。

STEP 5 Duration（速度・時間感）を決める

まずSpeedやSeconds（秒）で大きな見え方を作り、必要に応じてFrames（フレーム）で詰める。速すぎると読めず、遅すぎると間延びする。読みやすさ優先で。

ヒント：最初はSpeed/Secondsでざっくり、そのあとFramesで微調整。

STEP 6 Global Marginsで安全な範囲を保つ

Global Marginsで、画面端に対して安全に見える範囲を作る。テロップが端で切れたり、はみ出したりしないための運用上の設定。

STEP 7 開始・終了の空白とLoopを設計する

Blank Page on Start / End で始まりと終わりを整える（いきなり文字が入る／唐突に切れるのを防ぐ）。Loopは便利だが『止め方』まで設計する（本番でどう止めるか決めずにLoopしない）。Ease In / Outは気持ちよさより読みやすさで判断。

STEP 8 本番前に確認する

確認：最後まで読める速度か／端で切れず安全範囲に収まるか／開始と終了が唐突でないか／Loopする場合に意図通り止められるか／Per Scene

Lightingなどで見え方が変わらないか。差し替え（名前追加）しても崩れないかも見る。

本番前チェックリスト

- ☐ 『どう動かすか』より『どう直すか・運用するか』を先に決めた
- ☐ Scene GroupとSequenceのどちらで作るか選び、理由が説明できる
- ☐ 後から分かる名前を付けた
- ☐ 速度は読みやすさ優先（Speed/Seconds→Framesで調整）
- ☐ Global Marginsで端で切れない
- ☐ Loopの止め方・開始終了の空白まで設計した

9. Maskで見せる範囲を作る

隠す・抜く・見せる範囲を設計する

2 | 第14章 | レベル：中級 | 目安 25分 | 素材フォルダ：09_Mask

ねらい

四角い表示範囲（ワイプ／切り抜き）を作り、その範囲を動かせるようにする。『消す』ではなく『見せる範囲を決める』考え方で、本番で調整が要る値だけをPublishして運用を壊さない。

STEP 1 Maskの考え方を押さえる

Maskは『消す』のではなく『見せる範囲を決める』。何を隠して何を隠すかを先に決めてから作ると迷わない。使う前に『どの範囲を、いつ、どう見せたいか』を確認する。

STEP 2 Layer Objectで対象をまとめる

Maskをかけたい複数のObjectを Layer Object

でまとめると、まとめて範囲制御しやすい。まず対象をLayerに入れる。

STEP 3 Box Maskで四角い表示範囲を作る

Box Maskで四角形の表示範囲を作る。範囲の内側だけが見える。ワイプや切り抜き表示の基本形。画像のアルファで抜きたい場合はMask Material とアルファ素材を使う考え方もある。

ヒント： まずBox Maskで四角の範囲を確実に作ってから、必要ならアルファ素材の抜きへ進む。

STEP 4 Scene DirectorでMaskを動かす

Scene Directorで、Maskの範囲や位置を時間で動かせる（ワイプイン等）。範囲が動くことで『見えてくる／隠れる』演出になる。

STEP 5 調整が要る値だけPublishする

本番で調整する値だけをSequencerにPublishする。判断基準：本番中に本当に調整するか／制作調整用の値を本番データに残していないか／運用者が意味を理解できる項目名か／触ってはいけない値まで公開していないか／初期値に戻せるか／Take Inspectorで分かりやすいか。

STEP 6 本番前チェックと切り分け

確認：見せたい範囲だけが見え、隠したいものが隠れているか／範囲を動かしたとき破綻しないか／Publishした値が運用者に分かるか。うまくいかないときは、Layer構成・Mask種別・重なり順の順で切り分ける。

本番前チェックリスト

☐ 『消す』ではなく『見せる範囲を決める』で設計している

- ☐ 対象をLayer Objectでまとめた
- ☐ Box Maskで四角い表示範囲を作れた
- ☐ Scene Directorで範囲を動かせる
- ☐ 本番で調整する値だけをPublishし、触らせたくない値は隠した
- ☐ 初期値へ戻せる／項目名が運用者に分かる

10. Gradientで読みやすい色面を作る

装飾ではなく、現場で使える色面を設計する

2 | 第10章 | レベル：初級 | 目安 20分 | 素材フォルダ：10_Gradient

ねらい

文字が載る部分を少し暗くした『読ませるための座布団』や、端に向かって透明にして映像になじむパネルを作る。派手さではなく、コントロールできる色面を目的から設計する。

STEP 1 目的を先に決める

Gradientは4色のリニアグラデーションで、透明度も含められる。作る前に目的を決める：上を少し明るく下を暗く／文字の後ろだけ暗くして読みやすい座布団／端に向かって透明にして映像になじませる、など。『きれいな背景』とだけ考えない。

STEP 2 情報が載る場所を先に置く

色より先に、文字や数字が載る位置を決める。仮の文字を先に置いてから色面を作ると、あとで読みにくくならない。位置が決まらないまま色を作ると、文字を載せたとき破綻しやすい。

STEP 3 4色で役割を持たせる

4色は派手にするためではなく、明るさや色味の変化に役割を持たせるために使う。上下だけ明るさを変える、左右は同じにして縦方向の変化だけ作る、片側だけ暗くして文字置き場を作る、など。

STEP 4 透明度で『なじませる』と『読ませる』を両立

透明度にも役割を持たせる。端だけ透明にして映像になじませつつ、文字が載る部分は不透明寄りにして読ませる。角1か所だけ透明度を変えて抜け感を作る、といった使い方もできる。

ヒント： 透明度は『なじませる』と『読ませる』のバランス。文字の下は読める濃さを確保する。

STEP 5 少しずつ変えて確認する

設定は少しずつ変える。派手に作るより、コントロールできる状態で作るのが大事。ローワーサードなら文字の下を暗く、スコア表示なら数字が読める濃さ、チームカラーは強すぎない範囲でなじませる。

本番前チェックリスト

- ☐ 作る前に目的（読ませる/なじませる）を決めた
- ☐ 文字・数字の載る場所を先に置いてから色を作った
- ☐ 4色に役割を持たせた（変化の方向を絞った）
- ☐ 透明度で『なじませる』と『読ませる』を両立できている
- ☐ 文字の下が読める濃さになっている
- ☐ 派手さより、コントロールできる色面になっている

11. 現在時刻・カウントダウンを表示する

Date Timeで『変化する情報』として時間を扱う

4 | 第5章 | レベル：中級 | 目安 30分 | 素材フォルダ：11_Clock_Countdown

ねらい

現在時刻の表示と、試合・イベント開始までのカウントダウンを作る。時間を『文字』ではなく『変化する情報』として扱い、目標時刻を作って現在時刻との差を出し、見やすい文字列に整える。

STEP 1 時間を『変化する情報』として捉える

実務の時間表示（開始時刻・開始日・カウントダウン・曜日・一定時刻での切替）は『時間が変化する』前提で作る。完成した1枚の絵ではなく、値が変わっても成立する仕組みを作る。

STEP 2 ClockかDate Timeブロックかを切り分ける

現在時刻や日付をそのまま出すだけならClockで足りる。『目標時刻との差（カウントダウン）』『日時を分解して曜日を出す』『指定した年月日から日時を作る』などはDate Time系ブロックが要る。

STEP 3 現在時刻をそのまま出す（Clock）

まず現在時刻の表示から。Clockで現在時刻を取得し、Format（またはFormat Date Time）で見やすい文字列に整えてText Objectに出す。時・分・秒のうち必要な桁だけ出す。

STEP 4 目標時刻を作る（Encode Date Time）

カウントダウンには目標時刻が要る。Encode Date Timeで、年・月・日・時・分・秒から試合開始時刻などの目標日時を組み立てる。

ヒント：月末±1か月のような境界（1/31の1か月後など）は期待通りになるか実機で確認。

STEP 5 差を計算して残り時間を出す（Time Delta）

Clockで取った現在時刻と、Encodeで作った目標時刻をTime Deltaに渡して差を計算する。Days / Hours / Minutes / Secondsを使って残り時間を表示すれば、開始までのカウントダウンになる。

STEP 6 表示用に整える（Format Date Time）

Format Date TimeでDate Timeを表示用の文字列に整える。曜日が要るならDay of the Weekで取り出す。桁数が変わると幅が動くので、Text Objectの幅と揃えを確認する。

STEP 7 本番前に確認する

確認：現在時刻が正しく更新されるか／カウントダウンが0に近づくときの表示が破綻しないか／桁数が変わっても揃うか／タイムゾーンや基準時刻が意図通りか。値が変化し続ける前提で、しばらく置いて確認する。

本番前チェックリスト

- ☐ 時間を『変化する情報』として扱っている
- ☐ Clockで足りるか、Date Timeブロックが要るかを切り分けた
- ☐ Encode Date Timeで目標時刻を作れた
- ☐ Time Deltaで差を出し、残り時間を表示できた
- ☐ Format Date Timeで見やすく整え、桁揺れを確認した
- ☐ しばらく動かして更新・境界の破綻がないことを確認した